Real-time Anomaly Detection for Expiry Dates

Yaser Babukhan
1MDE1
University Of Galway
Galway, Ireland
m.babukhan1@universityofgalway.ie

Seán Kelly
1MECE1
University Of Galway
Galway, Ireland
s.kelly178@universityofgalway.ie

***Abstract*—This paper presents a single-class anomaly detection system for real-time inspection of date stamps on milk bottle caps, deployed on an NVIDIA Jetson Nano embedded platform. The approach requires only known-good samples for training, making it well-suited to manufacturing contexts where defect samples are scarce or defect modes are not known in advance.**

An image processing pipeline isolates the cap region from video frames, and the resulting crop is encoded as a 512-dimension feature vector by a frozen ResNet-18 CNN. PCA reduces this to 181 features while retaining 95% of variance, after which a multivariable Gaussian distribution is fitted using Ledoit-Wolf covariance regularisation to handle the limited amount of training data. Anomaly scoring uses Mahalanobis distance against the fitted distribution.

Evaluated on a custom dataset of 395 known-good and 10 defective caps, the system achieves an AUROC of 0.993 and 100% anomaly recall at the p_{99} threshold, with a false-positive rate of 3.5%. The full video inference pipeline runs at approximately 20 FPS on the Jetson Nano within a 10 W power budget.

***Index Terms*—Anomaly detection, date codes, ResNet, embedded, machine vision, Mahalanobis, Ledoit-Wolf, CNN, image processing**

I. Introduction

Automated quality control on production lines demands inspection systems that are fast, consistent, and cost-effective. These requirements lend themselves to the use of embedded machine vision specifically, as it can be built as an add-on to the existing production line, is often cheap on hardware (most hardware for embedded machine vision is under €1000), operates at low power, and utilises cameras capable of dealing with several items per second.

This project addresses the detection of defective date stamps on milk bottle caps, a task where defects (smeared ink, missing digits, print head misalignment, etc.) are rare, often visually subtle, and difficult to gather a large dataset of obvious failures. Additionally, these failures must be caught at production time, before the product is packed for shipping, otherwise the product will be wasted due to customer rejection upon delivery.

The project proposes a system for single-class anomaly detection. Rather than training a binary classifier that requires a balanced dataset of correct and incorrect sample images, the model learns a statistical description of acceptable date stamps only. Any cap whose date stamp features deviate from this learned distribution is then flagged as anomalous. This is well-suited to real-world manufacturing, where defect samples are often scarce and defect modes may not even be known in advance.

The pipeline combines a frozen, pre-trained convolutional image recognition network (ResNet-18 [1]) which converts images into numerical feature vectors, with a multivariable Gaussian distribution which models acceptable bottle caps. The model flags anomalies using Mahalanobis distance scoring to compare how far a given sample sits from the distribution against a threshold derived from the distribution. Both inference and distribution training run in real-time on an NVIDIA Jetson Nano, low-power system-on-module development board which is representative of hardware available for industrial embedded machine vision. The key design goals were:

- High anomaly recall with low false-alarm rate
- Real-time throughput (i.e. $\geq$30 FPS)
- Low-power system theoretically suited to production deployment (i.e. 5-10w power budget)

II. Background Research

A. Dataset

As existing datasets did not provide adequate coverage of the font, font size, and date stamp shape of the particular production line targetted by this project, a custom dataset was built by capturing images of bottle caps coming from the production line and preprocessing to give consistent size, crop, and orientation to the images in the dataset. The images were then sent through an image processing pipeline which isolates the bottle cap from the image, at which point the images were passed to the distribution training script. The dataset consists of 1025 augmented 256x256 crops of known good bottle caps, derived from the preprocessed images and expanded with photometric and geometric augmentations (gamma, saturation, brightness, and Gaussian noise).

Validation uses 395 known good and 57 intentionally defective crops. Because one-class learning only requires normal examples during training, collecting labelled anomalies is needed only for validation, which reduced the data acquisition burden significantly, which is a common motivation for anomaly-detection approaches in modern industrial inspection [5].

B. Feature Extraction

Collecting sufficient data to train a deep network from scratch is impractical for this purpose as previously outlined. Transfer learning from the ImageNet-pretrained [**?**] ResNet-18

model covers a sufficient amount of visual features without domain-specific supervised training. The final fully-connected layer (the head) is removed from the model, exposing the penultimate layer (a global average pooling layer) output: A 512-dimension feature vector describing the input image. The model backbone is frozen throughout, so no GPU memory is consumed by gradients and inference is very lightweight as the model does not back propagate. This is important for Jetson deployment where memory and GPU power are high for an embedded system, but not high enough to support back propagation and real-time inference at the same time. ResNet-18 has been shown to produce strong one-class baselines in prior work [4], [6], and also has seen extensive use on the Jetson Nano in open-source projects, so it was an obvious choice for this project. Producing 512 features for each image, it provides an extremely detailed description of each image, which can then be used to build an accurate distribution of known good bottle cap prints.

C. PCA Dimensionality Reduction

Not all 512 ResNet features are informative for bottle-cap stamps. A pure-NumPy incremental PCA projection is applied to the dataset after running it through ResNet but prior to building the distribution. Sufficient principal components are kept to explain 95% of training-set variance by default. This improves the covariance estimate that follows this step and also reduces inference compute which raises FPS. This ultimately boosts the accuracy of the system by discarding data which represents noise both in training and inference. This is consistent with the analysis of Defard et al. [4]'s research into Patch Distribution Modelling, an alternative approach to the same methods used in this project.

D. Ledoit-Wolf Gaussian Model

The training features are modelled as a multivariable Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\boldsymbol{\mu}$ is the mean feature vector of the training samples and $\boldsymbol{\Sigma}$ is the covariance matrix describing how much the features vary and correlate with one another. The covariance matrix acts as a multivariable standard deviation.

Reliably estimating $\boldsymbol{\Sigma}$ requires many multiples more samples than there are dimensions. With a 512-dimension feature vector coming from ResNet, and a limited training dataset, the covariance matrix Σ becomes "ill-conditioned", where its inverse becomes numerically unstable and fluctuates wildly with small changes to Σ. The inverse of Σ is required for calculating the Mahalanobis distance, which is how the whole system compares input samples to the distribution. A system needs to be implemented to resolve the numerical instability without increasing the size of the training dataset by several orders of magnitude.

Ledoit-Wolf shrinkage [2] addresses this by blending $\mathbf{S}$, the ill-conditioned covariance matrix, with a scaled identity matrix:

$$\hat{\boldsymbol{\Sigma}} = (1-\alpha)\mathbf{S} + \alpha \, \frac{\mathrm{tr}(\mathbf{S})}{d} \, \mathbf{I} \tag{1}$$

Here $\mathbf{S}$ is the raw covariance matrix, d is the feature dimensionality, $\mathbf{I}$ is the identity matrix, and $\frac{\mathrm{tr}(\mathbf{S})}{d}$ scales the identity to match the average variance of the data. tr is the "trace" of the matrix, which is the sum of values along its diagonal. The blend weight $\alpha \in [0, 1]$ controls how much the estimate is pulled toward this simpler, stable structure: $\alpha = 0$ gives the raw sample covariance while $\alpha = 1$ ignores the data entirely.

The resulting "shrunken" covariance matrix $\hat{\Sigma}$ isn't necessarily correct but *is* stable, in exchange for being slightly biased towards the identity matrix, which is acceptable because the entries which are not along the diagonal of the covariance matrix are typically the ones which are least reliable.

The Ledoit-Wolf method computes the optimal α analytically. Their paper shows that even though the true covariance is unknown, the error between $\hat{\Sigma}$ and the true covariance matrix can still be estimated analytically, and the optimal value of α can be calculated from these values only. This eliminates the need for cross-validation as is typically seen in neural network training, which would require a ridiculous number of validations due to the number of dimensions and the comparatively small size of the dataset, increasing training time by several orders of magnitude. This is the same regularisation strategy used in PaDiM [4] and related embedding-based methods of anomaly detection, specifically to avoid that issue.

E. Mahalanobis Distance Scoring

At inference, each cropped and preprocessed image is scored by its Mahalanobis distance to the training data distribution:

$$d(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \cdot \hat{\boldsymbol{\Sigma}}^{-1} (\mathbf{x} - \boldsymbol{\mu})} \tag{2}$$

where $\mathbf{x}$ is the feature vector of the image being tested, $\boldsymbol{\mu}$ is the mean feature vector of the known-good training samples, and $\hat{\boldsymbol{\Sigma}}^{-1}$ is the inverse of the regularised covariance matrix.

Mahalanobis distance is scale and correlation invariant. When the distribution being measured against is Gaussian, the Mahalanobis distance has a chi-squared distribution. This means interpretable percentile-based thresholds can be defined (e.g. p_{90}, p_{95}, p_{99}) which correspond to the probability that a good sample would have a distance greater than the Nth percentile of the training dataset. The approach is very much classical, dating back to 1936 [3], but it has proven more than suitable for dealing with single-class inspection problems like anomaly detection when dealing with large feature vectors from deep neural networks [6], [7].

III. Methodology

The pipeline is comprised of four standalone modules: preprocessing (`preprocess.py`), training (`build_distribution.py`), static inference (`inference.py`), and real-time inference (`video_inference.py`).

The preprocessing and static inference modules are both used in the training module, which builds the distribution, and the real-time inference pipeline which takes input from a camera and runs anomaly detection on the input feed.

a) Cap localisation and preprocessing: Raw images are passed through the cascaded bottle cap detection pipeline in `cap_detection.py`. The image is first downsampled to 500×500 px and illumination is normalised: if mean brightness falls below a threshold, a gamma lift ($\gamma = 0.4$) is applied, followed by CLAHE on the HSV data's value channel (clip limit 2.0, 8×8 tile grid).

Blob detection then operates in the HSV saturation space. Otsu's method determines an initial threshold, which is tried alongside fixed fallbacks (55, 65, 75, 85). For each threshold, a binary saturation mask is produced and cleaned with morphological close then open operations. Candidate contours are filtered by area (3-55% of image) and circularity, and the best is selected by a score combining circularity, proximity to the image centre, and log-area.

The true outer rim radius is then measured. For colour caps, a radial saturation walk samples 36 evenly-spaced angles outward from the detected centre, marking the last pixel still on the cap surface. For low-saturation caps a gradient-magnitude edge finder is used instead: the pixel with the highest Sobel gradient magnitude within a tight radial window is taken as the edge for each angle.

Finally, a 256×256 circular masked crop is extracted: the cap region is squared off, padded by 8% of the rim radius, resized to 256×256, and all pixels outside the circle mask are filled with black.

b) Stage 1b: Data augmentation: Each source image produces 12 augmented variants from `augment.py`. Augmentation parameters are decided independently for each variant, so the distribution of augmentations gives dense coverage near no transform and exponentially thinning probability toward extreme combinations, rather than a grid of fixed presets which would add in a large number of extreme transforms unlikely to match real-world inputs.

The colour augmentations operate in HSV space. Gamma correction, clipped to $[0.4, 2.5])$ models variation in overall exposure. Saturation scaling accounts for differences in colour rendering between cameras and environment lighting. An additive brightness shift covers small changes in ambient illumination e.g. due to sunlight.

Gaussian pixel noise (half-normal, $\sigma = 6$) and occasional mild Gaussian blur (25% probability, kernel size 3 or 5) model sensor noise and slight defocus respectively. Together these augmentations expand the training set from 171 source images to 1025.

c) Stage 2: Calibration: The 1025 known-good crops are fed through the frozen ResNet-18 backbone. The 512-dimensional global average-pooling output is retained, yielding a feature matrix of shape $(1025, 512)$.

PCA is then applied to the resulting data. The mean is subtracted and the data is projected onto the principal components that together explain 95% of variance, retaining 181 of 512 dimensions for the training dataset. This improves the conditioning of the subsequent covariance estimate and also reduces inference latency.

Ledoit-Wolf shrinkage is applied to the projected features to produce the regularised covariance matrix $\hat{\boldsymbol{\Sigma}}$. The precision matrix $\hat{\boldsymbol{\Sigma}}^{-1}$ is precomputed and stored alongside $\boldsymbol{\mu}$, the PCA projection, and the percentile thresholds p_{90}, p_{95}, p_{99} derived from the calibration-set Mahalanobis distances. The resulting distribution (`distribution.pkl`) is stored on disk.

d) Stage 3: Static inference: `inference.py` accepts a single image or a directory and scores each crop against the loaded calibration model. The cap localisation pipeline runs first; the resulting crop is projected through the stored PCA basis, the Mahalanobis distance computed, and the result compared against the chosen percentile threshold (p_{90}, p_{95}, or p_{99}, defaulting to p_{99}).

For each image, the raw Mahalanobis distance, the threshold value, and the margin (distance minus threshold) are printed. A positive margin indicates an anomaly; the magnitude indicates how far beyond the threshold the sample lies.

e) Stage 4: Real-time video inference: `video_inference.py` captures frames from a webcam or video file and runs the full detection and scoring pipeline on each frame. The cap localisation pipeline, feature extraction, PCA projection, and Mahalanobis scoring are all executed per-frame, with the result displayed in real time.

Detection noise causes the estimated cap centre and radius to jitter slightly between frames, which would produce a visually unstable crop even when the cap is stationary. An exponential moving average smooths the position and radius over time to help reduce this effect:

$$\hat{x}_t = \alpha\, x_t + (1 - \alpha)\, \hat{x}_{t-1} \tag{3}$$

where x_t is the current raw measurement and $\alpha = 0.7$ by default. Lower α gives a smoother but more lagged estimate; higher α responds faster but retains more jitter. When no cap is detected in a frame the smoothed state is reset, so stale positions do not persist when a cap moves out of frame, e.g. on a conveyor belt.

The display renders a coloured border and a semi-transparent banner showing the Mahalanobis distance, threshold, margin, and live FPS. The border and verdict label are green for a good cap, red for an anomaly, and yellow if no cap was detected in the frame.

IV. Testing and Results

A. Training and Calibration Performance

The training dataset Mahalanobis distances have a mean of 12.69 and a standard deviation of 1.71 (range of 8.50-22.42), with thresholds of $p_{90} = 14.96$, $p_{95} = 15.80$, $p_{99} = 17.60$. The Ledoit-Wolf shrinkage coefficient $\alpha = 0.0081$ is very small, indicating that after PCA reduction ($512 \rightarrow 181$ dimensions, $n = 1025$) the sample covariance is already well-conditioned and requires only minimal regularisation. This indicates that PCA was effective in helping stabilise the covariance matrix by reducing the number of dimensions present, and it also indicates that the number of dataset augmentations applied to the base dataset was appropriate,

TABLE I
VALIDATION DATASET PERFORMANCE

Metric	Value
Validation good samples	395
Validation anomalous samples	10
Anomaly detection recall (p_{99})	100% (10/10)
False-positive rate (p_{99})	3.5% (14/395)
AUROC	0.993
Ledoit-Wolf shrinkage α	0.0081
PCA components retained	181 of 512 (95% variance)

generating sufficient samples to prevent an ill-conditioned covariance matrix.

B. Embedded Performance

TABLE II
INFERENCE PERFORMANCE ON NVIDIA JETSON NANO

Metric	Value
Inference throughput (video pipeline)	≈20-25 FPS
Feature extraction latency (per frame)	≈35 ms
Cap detection latency (per frame)	≈15 ms
Overall inference latency	≈50 ms
Calibration model size	≈ 1 MB

C. Requirements Assessment

The system meets the primary embedded requirements: inference operates on a frozen model and precomputed distribution, requiring no on-device training. Training can be conducted on the embedded device if needed, utilising the Jetson Nano GPU. The real-time video inference pipeline runs at ≥20 FPS on the Jetson Nano hardware, utilising the GPU for ResNet-18 inference and running at an appropriate frame rate for use on a bottling line (max throughput 5 bottles/min). The percentile-based thresholding mechanism allows the sensitivity to be adjusted without retraining which is appropriate for a system of this type. Additionally, the power budget is more than respected, maxing out at 10w under normal operating conditions. Finally, the system has perfect recall. While the False Positive Rate is higher than desired, the rate is still low enough that it can be handled by the machine operator.

V. CONCLUSION

This project demonstrates that a frozen pre-trained CNN model, combined with an appropriate one-class statistical model (e.g Mahalanobis scoring, and Ledoit-Wolf covariance regularisation), results in a practical and production-deployable anomaly detection system, in this case for industrial date stamp inspection. The approach requires only "normal" samples for training, and is tolerant of small datasets through both covariance regularisation and dataset augmentation. The system achieves real-time inference within the power and memory budget of the NVIDIA Jetson Nano at an appropriate frame rate for this application.

The main limitations include sensitivity to distribution shift (e.g. significant changes in lighting), and poor recall for highly localised defects (e.g. a single missing digit) that produce only a small shift in the average-pooling layer of the CNN.

VI. INDIVIDUAL CONTRIBUTIONS

The report was written in equal proportions by both authors. The breakdown of Python code written by each author is as follows:

System / Module	Author(s)
Cap localisation pipeline (`cap_detection.py`, `preprocess.py`)	Yaser
Feature extraction & calibration (`features.py`, `build_distribution.py`)	Yaser
Statistical model: Ledoit-Wolf + Mahalanobis (`distribution.py`)	Seán
PCA module (`pca.py`)	Seán
Real-time video inference (`video_inference.py`)	Both
Static inference CLI (`inference.py`)	Seán
Data augmentation (`augment.py`)	Yaser

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, pp. 770–778, 2016.

[2] O. Ledoit and M. Wolf, "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[3] P. C. Mahalanobis, "On the Generalised Distance in Statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[4] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A Patch Distribution Modelling Framework for Anomaly Detection and Localisation," in *Proc. ICPR Workshops*, pp. 475–489, 2021.

[5] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in *Proc. IEEE CVPR*, pp. 9592–9600, 2019.

[6] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards Total Recall in Industrial Anomaly Detection," in *Proc. IEEE CVPR*, pp. 14318–14328, 2022.

[7] K. Lee, K. Lee, H. Lee, and J. Shin, "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks," in *Proc. NeurIPS*, pp. 7167–7177, 2018.